Анализ данных с использованием языка Python

Пояснения к коду. Семинар 9

Seaborn и Plotly: распределения, связи, множественные графики

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-09-seaborn-plotly.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинару 8, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/visualization.md`.

1 Группы на одном графике: hue

Ячейка 1. Подключение библиотек и таблица ДТП с часом

```
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
import seaborn as sns

sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
os.makedirs('img', exist_ok=True)  # папка для файлов
dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
↪ parse_dates=['datetime'])  # ДТП Брянской области
dtp['hour'] = dtp['datetime'].dt.hour  # час каждого ДТП
dtp.shape
```

- `import plotly.express as px` — новая библиотека: `plotly.express`, быстрые интерактивные графики plotly, под коротким именем `px`. Остальные строки подключения — как на семинаре 8.
- `dtp['hour'] = dtp['datetime'].dt.hour` — час каждого ДТП сразу столбцом таблицы: он нужен в нескольких графиках в форме «таблица и имя столбца».
- `dtp.shape` — печатается `(10114, 15)`: к 14 столбцам добавился час.

Ячейка 2. Пять значений освещения в три

```
kinds = {'Светлое время суток': 'светлое', 'Сумерки': 'сумерки',
         'В темное время суток, освещение включено': 'тёмное',
         'В темное время суток, освещение отсутствует': 'тёмное',
         'В темное время суток, освещение не включено': 'тёмное'}
dtp['light_kind'] = dtp['light'].map(kinds)  # пять значений в три
dtp['light_kind'].value_counts()
```

- `kinds = {...}` — словарь: слева от двоеточия значение из данных, справа то, во что его превратить. Три вида тёмного времени переводятся в одно «тёмное». Словарь записан на пяти строках: Python ждёт продолжения, пока фигурная скобка не закрыта. Значения из данных пишутся точно как в таблице: «темное» там без буквы «ё».

– `dtp['light'].map(kinds)` — заменяет каждое значение столбца по словарю; результат записывается новым столбцом `light_kind`. Значение, которого нет в словаре, стало бы пропуском.

– `value_counts()` — проверка: сколько ДТП в каждой из трёх групп.

Ячейка 3. Гистограмма часов со слоями по освещению

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(data=dtp, x='hour', hue='light_kind', hue_order=['светлое', 'сумерки', 'тёмное'],
             palette=['#c6dbef', '#6baed6', '#08519c'], multiple='stack', discrete=True, ax=ax)  # слои по освещению
ax.set_title('ДТП по часам суток и освещению')
ax.set_xlabel('Час')
ax.set_ylabel('Число ДТП')
ax.get_legend().set_title('Освещение');  # заголовок легенды
```

– `sns.histplot(data=dtp, x='hour', ...)` — гистограмма часов, как на семинаре 8; `discrete=True` — по столбцу на каждый час.

– `hue='light_kind'` — имя столбца, по значениям которого группы красятся разными цветами. Слово hue по-английски значит «оттенок».

– `hue_order=[...]` — порядок групп в легенде и в слоях. В стопке первая группа списка оказывается сверху, последняя — внизу: на рисунке тёмное время лежит в основании столбцов.

– `palette=[...]` — цвета групп в том же порядке, что в `hue_order`: светло-голубой, голубой, тёмно-синий.

– `multiple='stack'` — ставить слои стопкой, один на другой. Без этого seaborn наложил бы слои друг на друга полупрозрачно, и высоты читались бы неверно.

– `ax.get_legend().set_title('Освещение')` — `get_legend()` достаёт легенду рамки, а `set_title` меняет её заголовок: seaborn подписал бы легенду именем столбца, `light_kind`.

Ячейка 4. Водители со своими ДТП и пол

```
people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # участники ДТП
people['experience_clean'] = people['experience'].replace([95, 96, 99], np.nan)  # коды стажа пропусками
drivers = people[people['role'] == 'Водитель'].merge(dtp, left_on='dtp_id', right_index=True)  # водители с их ДТП
drivers['gender'].value_counts()
```

– Строки 1–3 — те же, что в ячейке 16 семинара 8: участники, служебные коды стажа пропуском, водители со своими ДТП.

– `drivers['gender'].value_counts()` — сколько водителей каждого пола: мужчин 12 752, женщин 1 916.

Ячейка 5. Плотность стажа по полу

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.kdeplot(data=drivers, x='experience_clean', hue='gender', ↩
↪ common_norm=False, ax=ax)  # плотность стажа по полу
ax.set_title('Стаж водителей: мужчины и женщины')
ax.set_xlabel('Стаж, лет')
ax.set_ylabel('Плотность')
ax.get_legend().set_title('Пол');
```

– `sns.kdeplot(...)` — кривая плотности: гистограмма, сглаженная в плавную линию; площадь под кривой равна единице. Сокращение kde — от kernel density estimate, «оценка плотности ядром».

– `hue='gender'` — своя кривая для каждого пола.

– `common_norm=False` — нормировать каждую группу отдельно: площадь под каждой кривой — единица. По умолчанию seaborn делит одну единицу на все группы по их размеру, и кривая маленькой группы почти ложится на ось.

– Пропуски стажа `kdeplot` пропускает сам.

Ячейка 6. Плотность часа: ловушка без ошибки

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.kdeplot(data=dtp, x='hour', ax=ax)  # плотность часа: ловушка
ax.set_title('Плотность часа ДТП: так не надо')
ax.set_xlabel('Час');
```

– `sns.kdeplot(data=dtp, x='hour', ax=ax)` — плотность целых чисел от 0 до 23. Сглаживание тянет кривую за края, от минус трёх до двадцати шести часов, и размывает пики. Для целых чисел — `histplot` с `discrete=True`.

2 Два числа: рассеяние и корреляция

Ячейка 7. Возраст машины и рассеяние

```
drivers['vehicle_age'] = drivers['datetime'].dt.year - ↩
↪ drivers['vehicle_year']  # возраст машины в годах
fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(data=drivers, x='vehicle_age', y='experience_clean', s=10, ↩
↪ alpha=0.2, ax=ax)  # точка на водителя
ax.set_title('Стаж водителя и возраст машины')
ax.set_xlabel('Возраст машины, лет')
ax.set_ylabel('Стаж, лет');
```

– `drivers['datetime'].dt.year - drivers['vehicle_year']` — год ДТП минус год выпуска машины, для каждой строки сразу; результат — новый столбец `vehicle_age`.

– `sns.scatterplot(...)` — диаграмма рассеяния seaborn: точка на каждую строку, `x` и `y` — имена столбцов. Строки с пропуском в одном из столбцов не рисуются.

– `s=10` — размер точки; `alpha=0.2` — прозрачность: на рисунке почти 14 тысяч точек, и без прозрачности они слились бы в пятно.

Ячейка 8. Коэффициент корреляции

```
drivers[['vehicle_age', 'experience_clean']].corr()  # корреляция двух столбцов
```

– `drivers[['vehicle_age', 'experience_clean']]` — таблица из двух столбцов: двойные квадратные скобки, внутри список имён.

– `.corr()` — коэффициенты корреляции всех пар столбцов этой таблицы. Результат — таблица 2 на 2: на диагонали единицы, корреляция столбца с самим собой, вне диагонали `-0.03022`. Строки с пропусками в паре столбцов не учитываются.

Ячейка 9. Рассеяние с линией тренда

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.regplot(data=drivers, x='vehicle_age', y='experience_clean',
            scatter_kws={'s': 10, 'alpha': 0.2}, line_kws={'color': 'black'}, ax=ax)  # точки и линия тренда
ax.set_title('Стаж водителя и возраст машины: линия тренда')
ax.set_xlabel('Возраст машины, лет')
ax.set_ylabel('Стаж, лет');
```

– `sns.regplot(...)` — рассеяние и прямая, проведённая через облако как можно ближе ко всем точкам; серая полоса вокруг прямой — её доверительный интервал.

– `scatter_kws={'s': 10, 'alpha': 0.2}` — словарь настроек для точек: размер и прозрачность, как в ячейке 7.

– `line_kws={'color': 'black'}` — словарь настроек для линии: чёрный цвет, чтобы линия была видна поверх синих точек.

Ячейка 10. Новички и возраст их машин

```
novice = drivers['experience_clean'] <= 2  # водители со стажем до двух лет
drivers.groupby(novice)['vehicle_age'].median()  # медиана возраста машины
```

– `drivers['experience_clean'] <= 2` — маска: `True` у водителей со стажем до двух лет включительно. У водителей без стажа в таблице будет `False`: сравнение пропуска с числом всегда ложно.

– `drivers.groupby(novice)` — группировка по маске: две группы, `False` и `True`. Группировать можно по Series той же длины, что и таблица.

– `['vehicle_age'].median()` — медиана возраста машины в каждой группе: `False 10.0`, `True 13.0`.

Ячейка 11. Корреляция по всей таблице: намеренная ошибка

```
dtp.corr()  # корреляция по всей таблице: ошибка
```

– `dtp.corr()` — корреляции по всем столбцам таблицы. В таблице есть текст, и pandas сообщает `ValueError: could not convert string to float: 'Выгоничский район'`: «не могу превратить строку в число». Ошибка намеренная.

Ячейка 12. Матрица корреляций числовых столбцов

```
numeric = ['participants_count', 'vehicles_count', 'dead_count', 'injured_count', 'road_defects', 'hour']
corr = dtp[numeric].corr()  # матрица корреляций
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr, cmap='coolwarm', center=0, annot=True, fmt='.2f', ax=ax)  # красное плюс, синее минус
ax.set_title('Корреляции числовых столбцов таблицы ДТП');
```

– `numeric = [...]` — список числовых столбцов, связь между которыми имеет смысл; координат в нём нет.

– `dtp[numeric].corr()` — таблица 6 на 6: корреляция каждой пары столбцов.

– `sns.heatmap(corr, ...)` — тепловая карта этой таблицы, как на семинаре 8.

– `cmap='coolwarm'` — расходящаяся палитра: синий — белый — красный. `center=0` — белый цвет приходится на ноль, синий — на минус, красный — на плюс.

– `annot=True` — числа в клетках; `fmt='.2f'` — два знака после точки.

3 Сводка внутри графика

Ячейка 13. Доля ДТП с погибшими по часам с полосой

```
dtp['fatal'] = dtp['dead_count'] > 0  # ДТП с погибшими
fig, ax = plt.subplots(figsize=(8, 4))
sns.lineplot(data=dtp, x='hour', y='fatal', ax=ax)  # доля по часам с полосой
ax.set_title('Доля ДТП с погибшими по часам суток')
ax.set_xlabel('Час')
ax.set_ylabel('Доля ДТП с погибшими');
```

– `dtp['dead_count'] > 0` — маска ДТП с погибшими, записанная столбцом `fatal`.

– `sns.lineplot(data=dtp, x='hour', y='fatal', ax=ax)` — линия по сырой таблице: для каждого часа seaborn сам считает среднее `y`. Среднее столбца из `True` и `False` — доля `True`, то есть доля ДТП с погибшими.

– Голубая полоса вокруг линии — 95-процентный доверительный интервал этой доли; seaborn рисует её сам. Чем меньше ДТП в часе, тем шире полоса.

Ячейка 14. Среднее погибших по числу участников с усами

```
few = dtp[dtp['participants_count'] <= 8]  # ДТП до восьми участников
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(data=few, x='participants_count', y='dead_count', ax=ax)  # среднее погибших с усами
ax.set_title('Погибших на одно ДТП по числу участников')
ax.set_xlabel('Участников')
ax.set_ylabel('Погибших на ДТП');
```

– `few = dtp[...]` — ДТП не больше чем с восемью участниками: с девятью и больше их единицы.

– `sns.barplot(data=few, x='participants_count', y='dead_count', ax=ax)` — столбцы средних, как на семинаре 8, но группы теперь по вертикальной оси: `x` — число участников, `y` — погибшие, высота столбца — среднее число погибших в ДТП с этим числом участников. Усы — 95-процентный доверительный интервал среднего.

4 Много линий и много панелей

Ячейка 15. Линия ДТП по месяцам на каждый год

```
dtp['year'] = dtp['datetime'].dt.year
dtp['month'] = dtp['datetime'].dt.month
monthly = dtp.groupby(['year', 'month']).size().reset_index(name='count')  # ДТП по годам и месяцам
fig, ax = plt.subplots(figsize=(9, 5))
sns.lineplot(data=monthly, x='month', y='count', hue='year', palette='viridis', legend='full', ax=ax)  # линия на год
ax.set_title('ДТП по месяцам в каждый год')
ax.set_xlabel('Месяц')
ax.set_ylabel('Число ДТП')
ax.get_legend().set_title('Год');
```

– Строки 1–2 — год и месяц столбцами таблицы.

– `dtp.groupby(['year', 'month'])` — группы по двум столбцам сразу: каждая пара «год, месяц» — своя группа; `.size()` — число ДТП в каждой. Результат — Series, у которого в индексе две части, год и месяц.

– `.reset_index(name='count')` — превращает индекс в обычные столбцы: получается таблица из трёх столбцов `year`, `month` и `count`, где `name` задаёт имя столбца с числами. Функциям, которые принимают таблицу и имена столбцов, нужна именно такая таблица.

– `sns.lineplot(data=monthly, x='month', y='count', hue='year', ...)` — линия на каждый год; в `monthly` на каждую пару одно число, поэтому полосы нет.

– `palette='viridis'` — палитра-шкала от тёмно-фиолетового к жёлтому: ранние годы тёмные, поздние жёлтые.

– `legend='full'` — показать в легенде все годы. По умолчанию для чисел seaborn показывает в легенде только несколько значений шкалы.

Ячейка 16. Гистограммы часов по дням недели: панели

```
dtp['weekday'] = dtp['datetime'].dt.day_of_week  # понедельник 0
g = sns.displot(data=dtp, x='hour', col='weekday', col_wrap=4, ↩
↪ discrete=True, height=2.5)  # панель на день
g.set_titles('День недели {col_name}')
g.set_axis_labels('Час', 'Число ДТП')
g.figure.suptitle('ДТП по часам в каждый день недели, понедельник 0')
g.figure.tight_layout();
```

– `dt.day_of_week` — номер дня недели, 0 — понедельник.

– `sns.displot(...)` — распределения на панелях. Функция сама создаёт фигуру и рамки, поэтому `ax` ей не передают и `plt.subplots` не нужен.

– `col='weekday'` — отдельная панель для каждого значения столбца, то есть для каждого дня; `col_wrap=4` — после четырёх панелей перейти на новый ряд; `height=2.5` — высота каждой панели в дюймах.

– `g = ...` — результат не оси, а сетка панелей `FacetGrid`. Подписи ставятся её методами: `set_titles` пишет заголовок каждой панели, вместо `{col_name}` подставляется значение дня; `set_axis_labels` подписывает оси всех панелей сразу.

– `g.figure` — фигура, на которой лежит сетка: общий заголовок `suptitle` и `tight_layout` ставятся ей.

Ячейка 17. Ящики стажа по тяжести для мужчин и женщин

```
order = ['Легкий', 'Тяжёлый', 'С погибшими']
g = sns.catplot(data=drivers, x='severity', y='experience_clean', ↩
↪ kind='box', order=order,
                col='gender', height=4, aspect=1.1)  # ящики по полу
g.set_titles('{col_name} пол')
g.set_axis_labels('Тяжесть ДТП', 'Стаж, лет')
g.figure.suptitle('Стаж водителей по тяжести ДТП')
g.figure.tight_layout();
```

– `sns.catplot(...)` — графики по категориям на панелях, как `displot` для распределений. `kind='box'` — вид графика, ящики с усами; `order` — порядок ящиков, как в ячейке 17 семинара 8.

– `col='gender'` — панель на каждый пол; `height=4` — высота панели, `aspect=1.1` — ширина в 1.1 раза больше высоты.

– `g.set_titles('{col_name} пол')` — заголовки панелей «Женский пол» и «Мужской пол».

Ячейка 18. Все пары трёх числовых столбцов

```
three = drivers[['experience_clean', 'vehicle_age', ↩
↪ 'violations_count']].dropna()  # три числовых столбца
g = sns.pairplot(three, plot_kws={'s': 8, 'alpha': 0.2}, height=2.5)  # ↩
↪ все пары столбцов
g.figure.suptitle('Стаж, возраст машины и число нарушений')
g.figure.tight_layout();
```

– `drivers[[...]].dropna()` — три столбца без строк с пропусками: `pairplot` строки с пропусками не принимает.

– `sns.pairplot(three, ...)` — сетка 3 на 3: на пересечении двух разных столбцов рассеяние, на диагонали гистограмма столбца.

– `plot_kws={'s': 8, 'alpha': 0.2}` — настройки точек рассеяния: размер и прозрачность.

5 Plotly

Ячейка 19. Интерактивные линии по годам

```
fig = px.line(monthly, x='month', y='count', color='year', markers=True, title='ДТП по месяцам и годам')  # интерактивные линии
fig.update_layout(xaxis_title='Месяц', yaxis_title='Число ДТП', legend_title_text='Год')
fig
```

– `px.line(monthly, ...)` — интерактивные линии plotly. Первый аргумент — таблица, `x` и `y` — имена её столбцов; поэтому нужна таблица `monthly` из ячейки 15, а не сводка с индексом.

– `color='year'` — отдельная линия своего цвета на каждый год, как `hue` в seaborn. Цвета берутся по кругу из десяти, поэтому у 2015 и 2025 года цвет совпадает.

– `markers=True` — точки на линиях; `title` — заголовок.

– `fig.update_layout(...)` — подписи осей `xaxis_title` и `yaxis_title` и заголовок легенды `legend_title_text`; без них plotly подписал бы оси и легенду именами столбцов `month`, `count` и `year`.

– `fig` — имя фигуры в последней строке: Jupyter показывает её интерактивной. То же делает `fig.show()`.

Ячейка 20. Столбцы с подсказкой

```
top6 = dtp['category'].value_counts().head(6).reset_index()  # таблица: вид, число
fig = px.bar(top6, x='count', y='category', orientation='h', title='Шесть самых частых видов ДТП')  # столбцы с подсказкой
fig.update_layout(xaxis_title='Число ДТП', yaxis_title='', yaxis={'categoryorder': 'total ascending'})
fig
```

– `value_counts().head(6).reset_index()` — шесть самых частых видов ДТП, превращённые в таблицу из двух столбцов: `category` и `count`.

– `px.bar(top6, x='count', y='category', orientation='h', ...)` — столбцы plotly; `orientation='h'` — горизонтальные: длина по оси X, подписи по оси Y.

– `yaxis={'categoryorder': 'total ascending'}` — порядок видов по оси Y по возрастанию числа. Ось Y plotly идёт снизу вверх, поэтому самый частый вид оказывается сверху.

Ячейка 21. Сохранение интерактивного графика в HTML

```
1 fig = px.line(monthly, x='month', y='count', color='year', markers=True, ↩
  ↪ title='ДТП по месяцам и годам')
2 fig.update_layout(xaxis_title='Месяц', yaxis_title='Число ДТП', ↩
  ↪ legend_title_text='Год')
3 fig.write_html('img/dtp-by-month.html', include_plotlyjs='cdn')  # файл ↩
  ↪ для браузера
4 os.path.getsize('img/dtp-by-month.html')
```

– Строки 1–2 — та же фигура с теми же подписями, что в ячейке 19: в ячейке 20 переменная `fig` была занята столбцами, и фигуру строят заново.

– `fig.write_html('img/dtp-by-month.html', ...)` — сохраняет фигуру в файл HTML, страницу для браузера; подсказки и легенда в ней работают без Python.

– `include_plotlyjs='cdn'` — не встраивать библиотеку plotly в файл, а подключать её из интернета при открытии. Файл весит около 13 тысяч байт вместо почти пяти миллионов, зато без интернета график в нём не появится.

– `os.path.getsize(...)` — размер файла в байтах.

6 Карта

Ячейка 22. ДТП на карте с подложкой

```
1 inside = dtp[dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, ↩
  ↪ 35.5)]  # ДТП внутри рамки области
2 colors = {'Легкий': '#a6cee3', 'Тяжёлый': '#1f78b4', 'С погибшими': ↩
  ↪ '#e31a1c', 'Без пострадавших (нет данных)': '#999999'}
3 fig = px.scatter_map(inside, lat='lat', lon='lon', color='severity', ↩
  ↪ color_discrete_map=colors,
4                       hover_data=['district', 'datetime'], ↩
  ↪ labels={'severity': 'Тяжесть'}, zoom=7, height=600, title='ДТП Брянской ↩
  ↪ области')  # точки на карте
5 fig
```

– `inside = dtp[...]` — ДТП внутри рамки области, как в ячейке 21 семинара 8.

– `colors = {...}` — словарь «группа — цвет»: ДТП с погибшими красные, остальные спокойных цветов.

– `px.scatter_map(inside, lat='lat', lon='lon', ...)` — точки на карте с подложкой: `lat` — столбец широты, `lon` — столбец долготы. Подложка грузится из интернета.

– `color='severity'` — цвет по тяжести; `color_discrete_map=colors` — какой цвет у какой группы. Без словаря plotly раздал бы цвета по порядку появления групп в таблице.

– `hover_data=['district', 'datetime']` — что ещё показать в подсказке при наведении: район и дату.

– `labels={'severity': 'Тяжесть'}` — словарь «столбец — подпись»: в легенде и подсказке вместо имени столбца `severity` будет «Тяжесть».

– `zoom=7` — начальный масштаб; `height=600` — высота карты в пикселях.

Ячейка 23. Перепутанные координаты: ловушка без ошибки

```
fig = px.scatter_map(inside, lat='lon', lon='lat', zoom=3, height=400, ↩
↪ title='Широта и долгота перепутаны')  # ловушка
fig
```

– `lat='lon', lon='lat'` — в широту передан столбец долготы, в долготу — столбец широты. Ошибки нет: и там и там числа в допустимых пределах, и точки уезжают в Иран, около 34 градусов северной широты и 53 восточной долготы.